Передача за значенням і керування 74HC595

Урок 2 — 2МАК, SST-INF-01c — Функції, пам'ять, `#define` і перша програма для 595

22/10/2026

Примітка

На цьому уроці ми повертаємося до **функцій** **з** **параметрами** з роздаткового матеріалу A01 і дивимося, **що** **відбувається** **в** **пам'яті**, коли функція отримує число: це **передача** **за** **значенням** (*passaggio per valore*). Потім переходимо до практики: збираємо в **Tinkercad** схему з **74HC595** і 8 світлодіодами (ту саму, що в матеріалі A04 з електроніки) і крок за кроком пишемо програму, яка нею **керує**, аж поки не скоротимо її до **циклу** **`for`** на кілька рядків.

1 Повторення: функція `lampeggio`

Ми казали, що **функція** схожа на **макрос**: це **набір** **інструкцій**, зібраних під **одним** **ім'ям**. Коли я викликаю `lampeggio` ("блимання"), Arduino виконує всі інструкції всередині. Минулого разу ми написали функцію з трьома **параметрами**: **вивід** (*piedino*), яким блимати, **кількість** **блимань** і **час** кожної фази (тобто швидкість блимання).

```
void lampeggio(int pin, int n, int t)
{
  for (int i = 0; i < n; i++)
  {
    digitalWrite(pin, HIGH);
    delay(t);
    digitalWrite(pin, LOW);
    delay(t);
  }
}

void loop()
{
  // вивід 5: 3 блимання по 200 мс
  lampeggio(5, 3, 200);
  // вивід 12: 7 блимань по 500 мс
  lampeggio(12, 7, 500);
}
```

Функція **підлаштовується** під параметри, які ми їй передаємо: не переписуючи щоразу код, я викликаю скільки завгодно блимань і кажу, **який** **вивід**, **скільки** **разів** і **з** **яким** **часом**. Цей приклад належить до теми **функцій** **з** **передачею** **за** **значенням**.

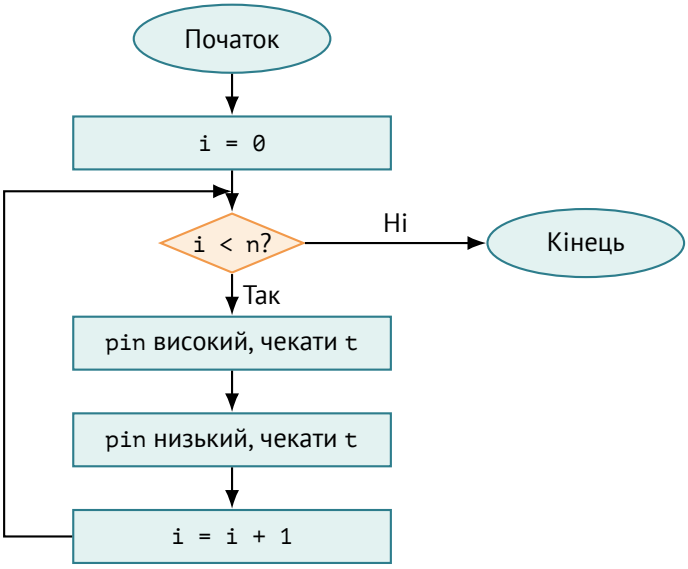


Рис. 1: Блок-схема функції `lampeggio(pin, n, t)`.

2 Що означає “передача за значенням”

Визначення

При **передачі** **за** **значенням** той, хто викликає функцію, дає їй **копію** числа, а не саму змінну. Функція працює **з** **копією**.

На дошці був заголовок: *“Функції з передачею за значенням: робиться копія параметра”* (*Funzioni con passaggio per valore: effettua una copia del parametro*). Щоб зрозуміти, що це означає насправді, спершу подивимося, **як** **влаштована** **пам’ять**.

2.1 Як фізично влаштована оперативна пам’ять (RAM)

У комп’ютері дані програми зберігаються в **RAM**. **Модуль** **RAM** — це невелика плата (PCB) з багатьма **контактами**, надрукованими по одному краю: замість ніжок, як у мікросхем, які ми використовуємо, у неї **позолочені** **доріжки**, що вставляються в слот материнської плати. Часто модуль накритий **металевою** **пластиною** (іноді з кольоровими вогниками “для краси”, але це лише естетика): пластина потрібна, щоб **відводити** **тепло**, бо RAM працює **дуже** **швидко**, а те, що працює швидко, завжди **гріється**.

Якщо зняти пластину, видно кілька **чорних** **мікросхем** з одного і з другого боку. Порахуймо для типового модуля на **16** **ГБ**: якщо там 4 мікросхеми з одного боку і 4 з другого, разом **8** **мікросхем**, то кожна має **2** **ГБ** ($8 \times 2 = 16$). Модуль на 8 ГБ матиме 8 мікросхем по 1 ГБ, на 32 ГБ — 8 мікросхем по 4 ГБ, або більше менших мікросхем: це залежить від того, як вирішив **виробник**, залежно від того, що йому дешевше.

Ці мікросхеми не мають наскрізних “ніжок”: це компоненти **SMD** (*Surface Mount Device*, поверхневий монтаж), яким **не** **потрібні** **отвори**; їх паяють, **поклавши** **зверху** на плату. Якщо відкрити мікросхему, всередині є **кремнієвий** **кристал**, набагато менший за чорний корпус: корпус великий лише для того, щоб його було **зручно** **тримати**. Кристал з’єднаний із зовнішніми виводами **тонесенькими** **золотими** **дротиками**. Дехто намагається видобувати золото зі схем, але з цілої палети комп’ютерів виходить лише 2–3 грами: це не варте зусиль.

Усередині кристала пам’ять — це **велика** **сітка** клітинок, кожна з яких може містити **1** **або** **0**. У 2 ГБ є 2 мільярди байтів, тобто приблизно **16** **мільярдів** **бітів**: 16 мільярдів клітинок. Можна уявити їх як **16** **мільярдів** **тригерів** (*flip-flop*, пристрої, які “тримають пам’ять”, ми бачили їх на електроніці); насправді в RAM комп’ютерів технологія інша: кожен біт зберігається в крихітному **конденсаторі**, компоненті, який **тримає** **електричний** **заряд**. Але принцип той самий: це **елемент** **пам’яті**.

2.2 Пам’ять як комод із шухлядами

Малювати 16 мільярдів клітинок нам не треба: уявімо пам’ять як **комод** **із** **шухлядами** (*cassettiera*). Згадайте “тижневик”, комод із шухлядою на кожен день тижня. Кожна шухляда має **номер**: шухляда 0, 1, 2, 3, … аж приблизно до 2 мільярдів. Кожна шухляда — це **байт**, поділений на **8** **бітів**, і кожен із цих 8 бітів можна записати.



Рис. 2: Пам’ять як нумерований комод із шухлядами (з малюнка на дошці).

Коли в програмі ми **оголошуємо** **змінну**, наприклад `char x;`, машина **вибирає** **вільну** **шухляду**

(скажімо, номер 6) і чіпляє на неї **мітку** `x`. Якщо перезапустити програму, наступного разу `x` може опинитися в шухляді 422 або 1037: місце **змінюється** **при** **кожному** **запуску**. Насправді є **алгоритм**, який вирішує, але для нас це ніби **випадково**. `int` на Arduino займає **2** **байти** (16 бітів), тобто **дві** **сусідні** **шухляди**.

Повернімося до `lampeggio(5, 3, 200)`. Число 5 у виклику десь лежить у пам’яті. Коли функція стартує, параметр `int pin` **створюється** в якійсь шухляді, наприклад у 3: це ціле число, воно займає 16 бітів, тобто **дві** **шухляди**, 3 і 4. Туди **копіюється** значення 5. Я навмисно кажу **скопійовано** і **продубльовано**: `pin` — не оригінальне 5, а **копія**. Для `lampeggio` це не створює жодної проблеми: функція займає трохи більше пам’яті, але для нас це дрібниця.

2.3 Приклад `duplica`: функція працює з копією

Поки функція лише *використовує* число (як `lampeggio`, що вибирає за ним вивід), те, що вона працює з копією, нам не заважає. Проблема виникає, коли функція **змінює** **дані**. Спробуймо функцію, яка мала б **подвоїти** число (*duplica* = “подвоїти”):

```
void loop()
{
  int x;
  x = 5;
  duplica(x);
  // скільки тут x? Досі 5!
}

void duplica(int pippo)
{
  pippo = pippo * 2;
}
```

Простежимо, що відбувається в комоді:

1. `int x;` створює `x` у якійсь шухляді; `x = 5;` записує туди **5**.
2. `duplica(x)` викликає функцію: створюється змінна `pippo` **в** **іншій** **шухляді** (два байти, бо це `int`), і туди **копіюється** 5.
3. `pippo = pippo * 2;` бере 5 з `pippo`, множить на 2 і записує **10** у шухляду `pippo`.
4. На **закривальній** **фігурній** **дужці** функція закінчується і **“знищує”** свої змінні.

Скільки в кінці `x`? **Досі** **5**: функція працювала **з** **копією**, а не з оригінальною `x`.

Примітка

“Знищити” змінну — слово трохи неточне, але передає ідею. Коли функція закінчується, вона **звільняє** (*dealloca*) свої змінні: **видаляє** **посилання**, тобто забуває, що `pippo` була в тій шухляді. Число 10 **там** **лишається**, але шухляда більше не зарезервована: вона **вільна** для іншої змінної, яка рано чи пізно запише туди щось своє. Це як місце, зайняте для однокласника, поки він щось робить: коли він закінчив, місце знову вільне, навіть якщо він лишив там рюкзак. Тому, якщо в `loop` запитати, скільки `pippo`, то **`pippo`** **не** **існує**: вона існує лише **під** **час** **виконання** функції `duplica`.

2.4 Отримати результат через `return`

Щоб функція **повернула** обчислене значення, треба змінити дві речі: функція вже не `void`, а оголошує **тип значення, що повертається** (`int`), і в кінці використовує **`return`**. У виклику результат **присвоюють** змінній:

```
void loop()
{
  int x;
  x = 5;
  x = duplica(x);
  // тепер x дорівнює 10
}

int duplica(int pippo)
{
  pippo = pippo * 2;
  return pippo;
}
```

Тепер послідовність така: створюється `pippo` (в іншій шухляді: **щоразу інакше**), туди копіюється 5, перезаписується на 10, а `return pippo` каже: “коли закінчиш обчислення, **значенням усього цього блоку** має бути значення `pippo`”. 10 копіюється в `x`, потім `pippo` звільняється: але **потрібне значення** ми **отримали**.

Увага

Підсумок: при передачі за значенням, якщо функція **не повинна давати результат** (вона `void`), копія нам не заважає. Якщо ж дані треба **обробити і повернути**, використовуємо **значення, що повертається** (`return`) і **присвоюємо** його змінній у виклику.

2.5 Конкретніший приклад: вага куба, надрукованого на 3D-принтері

Слайсер (*slicer*) 3D-принтера, крім підготовки файлу до друку, каже, **скільки філаменту** потрібно: якщо дати йому куб, він скаже, наприклад, що треба 15–20 грамів матеріалу. Як він це робить? Він знає **питому вагу** матеріалу (скільки важить кубічний сантиметр) і вміє рахувати **об’єм** предмета: **вага** = **питома вага** × **об’єм**. Для куба об’єм — це сторона × сторона × сторона.

Для прикладу беремо питому вагу **2 грами на кубічний сантиметр**. Одиниці виміру сходяться: $\frac{\text{г}}{\text{см}^3} \times \text{см}^3 = \text{г}$, тобто при множенні на об’єм (у см^3) кубічний сантиметр “у знаменнику” скорочується і лишаються **грами**.

Примітка

Значення 2 г/см^3 зручне, щоб рахувати в голові. Справжня питома вага **PLA** — приблизно 1,24 г/см^3.

```
void loop()
{
  // сторона куба в сантиметрах
  int lato = 5;
  int peso = calcolaPeso(lato);
}

int calcolaPeso(int x)
{
  int volume = x * x * x;
  int peso = 2 * volume;
  return peso;
}
```

(`lato` = сторона, `peso` = вага, `volume` = об’єм, `calcolaPeso` = “обчисли вагу”.) Простежимо в комоді, як на дошці:

1. `int lato = 5;` створює `lato` у шухляді і кладе туди **5**.
2. `int peso = calcolaPeso(lato);` створює змінну `peso` у `loop` (на дошці **червону**); її значення залежить від функції, тому вона “**в очікуванні**”, поки функція не закінчить.
3. Заходимо в `calcolaPeso`: створюється `x` в іншій шухляді, і туди **копіюється** значення `lato`, тобто 5.
4. `int volume = x * x * x;` створює `volume` і записує $5 \times 5 = 25$, $25 \times 5 = \mathbf{125}$. У вибраній шухляді вже було старе число, але “яка різниця”: воно **перезаписується**.
5. `int peso = 2 * volume;` створює **ще одну** змінну `peso` (на дошці **синю**), в іншій шухляді, ніж та, що в `loop`: у них **однакове ім’я**, але це **дві зовсім різні змінні**, які ніяк не пов’язані. Туди йде $2 \times 125 = \mathbf{250}$.
6. `return peso;` бере значення **синьої** `peso` і робить його значенням усього виклику `calcolaPeso(lato)`: 250 копіюється в **червону** `peso`.
7. Функція закінчується і **звільняє** `x`, `volume` і синю `peso`. Числа лишаються в шухлядах, але їх перезапише будь-що інше, що станеться в програмі. Лишаються `lato` (5) і червона `peso`, яка тепер має **очікуваний результат**: 250 грамів.

Увага

Типова помилка: повторюючи хід, на кроці 5 хочеться записати 250 у `peso` **з loop**. Насправді `int peso` всередині функції створює **нову змінну** (синю) в іншій шухляді: червона `peso` отримує значення **лише через `return`**, у кінці.

Примітка

Ім’я параметра (`x`) може бути **іншим**, ніж ім’я переданої змінної (`lato`): функція отримує лише **значення**. Так само змінна, створена у функції, може мати **те саме ім’я**, що й змінна в `loop`: це два об’єкти з однаковим ім’ям, але **в різних місцях**. А якщо викликати функцію знову, `volume` **не обов’язково** потрапить у ту саму шухляду, що й раніше.

3 Схема з 595 у Tinkercad

Переходимо до практики. Схема — та сама, що в матеріалі **A04** **з** **електроніки** (“Pilotare il 74HC595”): **595**, з’єднаний із **8** **світлодіодами**, кожен зі своїм резистором. У KiCad ми поки не продовжуємо розведення і виготовлення, бо **верстат** **на** **обслуговуванні**; тим часом **відтворюємо** **схему** **в** **Tinkercad** і пишемо firmware.

595 у Tinkercad знаходиться, якщо відкрити **усі** **компоненти**: внизу списку є **регістр** **зсуву** (*registro a scorrimento*), або можна пошукати “595”. За схемою постійні з’єднання такі:

- вивід **8** (GND) до **маси**;
- вивід **16** (VCC) до **5 В** (тобто до червоної шини макетної плати, з’єднаної з 5 В Arduino);
- вивід **10** (SRCLR) до **5 В**, щоб регістр ніколи не очищався;
- вивід **13** (OE) до **GND**, щоб виходи були завжди ввімкнені;
- **8** **виходів** до світлодіодів, кожен зі своїм **резистором** до маси.

У схемі KiCad Arduino не намальований: замість нього є **роз’єм** **J1** (`Conn_01x05`, 1 ряд з 5 контактів), куди підуть дроти до Arduino. У Tinkercad Arduino є, і дроти, які на схемі йдуть до J1, ідуть прямо до його виводів. Три дроти до Arduino можна під’єднати до будь-яких виводів: вибирати їх якось особливо не треба. На уроці схема така:

Arduino	вивід 595	назва
3 (помаранчевий дріт)	14	SER (дані)
5 (жовтий дріт)	12	RCLK (виводить на виходи)
7 (зелений дріт)	11	SRCLK (зсуває біт)

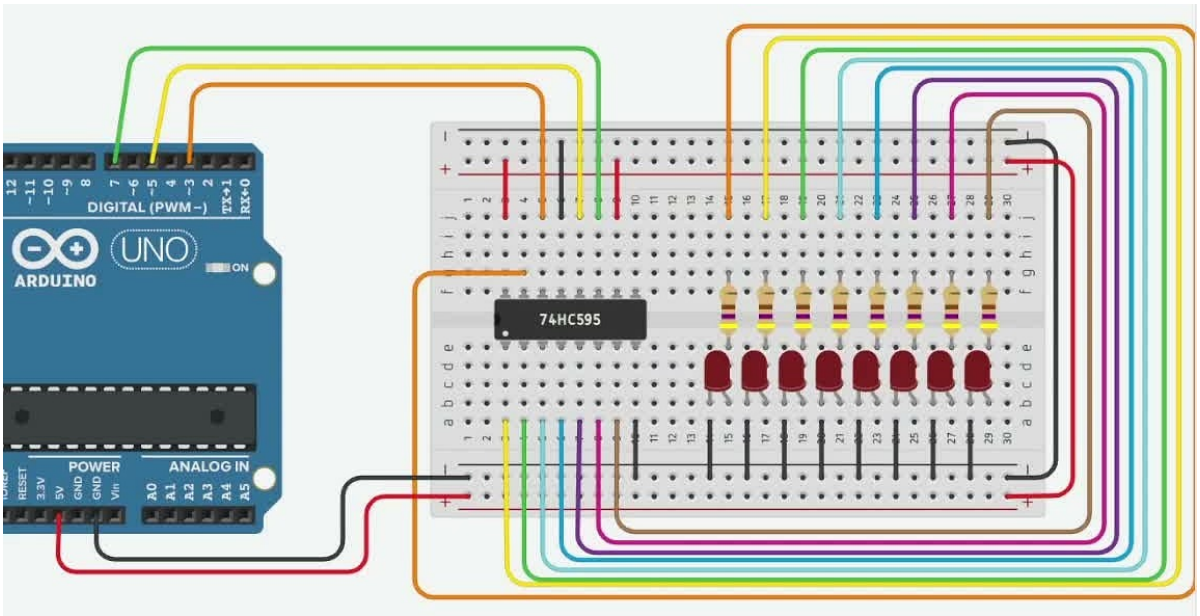


Рис. 3: Готова схема в Tinkercad: Arduino (виводи 3, 5, 7) з’єднаний із 74HC595, який керує 8 червоними світлодіодами з їхніми резисторами. Дроти проведено “зовні”, щоб уникнути перетинів червоного і чорного і зробити малюнок зрозумілішим.

Увага

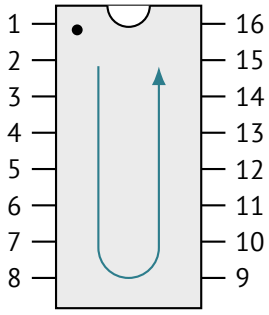
Під час малювання два дроти були проведені **на одну клітинку лівіше**, ніж треба, і потім виправлені. Перед симуляцією **перевірте кожен дріт** за схемою: дроту, зсунутого лише на один отвір макетної плати, досить, щоб схема не працювала.

3.1 Як нумерують виводи мікросхеми

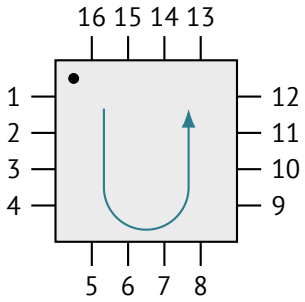
Tinkercad дає виводам 595 назви, **не такі**, як у KiCad (наприклад, якщо навести мишку на вивід 14, з'являється "ingresso", тобто "вхід"): щоб знати, що це за вивід, краще міркувати за його **номером**. А номер завжди визначається за тим самим правилом, без довідника.

Визначення

Дивимося на мікросхему **згори**, **виїмкою** (півмісяцем) догори. **Вивід 1** — той, що **одразу ліворуч від виїмки** (часто позначений ще й крапкою). Далі виводи рахують **проти годинникової стрілки**: вниз по лівому боці, вгору по правому.



DIP на 16 виводів



мікросхема з виводами на 4 боках

Рис. 4: Правило нумерації: від виводу 1, біля виїмки або крапки, проти годинникової стрілки, дивлячись на мікросхему згори. Діє і для мікросхем з виводами на чотирьох боках.

Увага

Типова помилка — нумерувати правий бік **згори вниз** (9, 10, 11, 12… починаючи згори): це не узгоджується з правилом для лівого боку. Якщо рахувати **проти годинникової стрілки**, після 8 (внизу ліворуч) переходимо до **9 внизу праворуч** і **піднімаємося** до 16, який стоїть **навпроти 1**, по другий бік виїмки. Це правило ми бачили ще минулого року, і воно діє для всіх мікросхем.

Застосуймо правило до 595: помаранчевий дріт (вивід 3 Arduino) приходить на вивід **14**, це **SER**; жовтий (вивід 5) — на **12**, це **RCLK**; зелений (вивід 7), методом виключення, — на **11**, **SRCLK**.

4 Мітки: #define

4.1 Проблема: пам'ятати номери виводів

У `setup()` ми пишемо **налаштування**: три виводи керують зовнішнім компонентом, тож усі три — **виходи**.

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(7, OUTPUT);
}
```

Якщо ви вибрали інші виводи, код, звісно, треба змінити. Але є проблема: ці числа ми будемо **використовувати** **знову** в усьому коді, щоб вирішувати, що піднімати і що опускати, і щоразу маємо **пам'ятати**, що 3 — це SER, 5 — RCLK, а 7 — SRCLK. З трьома дротами це вже незручно; уявіть схему з **72** **дротами**, що в деяких проєктах справді буває.

Примітка

Викладач пише код у **Visual** **Studio** **Code**, у файлі `.cpp`, лише тому, що редактор **розфарбовує** синтаксис і код легше читати; потім копіює його в Tinkercad. VS Code показує деякі **помилки**, бо не знає команд Arduino: там вони не важливі. Ви пишете прямо в редакторі коду Tinkercad (у режимі **Текст**).

4.2 Ідея: "знайти і замінити"

Візьмімо текст (згенерований штучним інтелектом за дві секунди, просто для прикладу): "Сонячне світло освітлювало кімнату… і той старий *будинок* ніби оживав щоранку", і уявімо, що він на **десятки** **сторінок**. Якщо ми вирішимо, що це був не будинок, а *хатинка*, довелося б вручну шукати кожен "будинок", який, може, повторюється 72 рази. Натомість текстові редактори на зразок Word мають команду **знайти** **і** **замінити**: "знайти *будинок*, замінити на *хатинка*, замінити все". Можна навіть замінити "хатинка" на число 72: у тексті це не має сенсу, але ідея та сама. І заміна робиться **до** **того**, як текст віддають тому, хто його читатиме. (Інший приклад: я копіюю твір однокласника "як я провів літо" і через "знайти і замінити" міняю його ім'я на своє… ідея зрозуміла.)

У C можна зробити саме це за допомогою **`#define`**: створюємо **мітку** (*etichetta*, *label*), і щоразу, коли компілятор її знаходить, він **замінює** її тим, що ми написали, **до** **компіляції**.

```
#define SER 3
#define RCLK 5
#define SRCLK 7

void setup()
{
  pinMode(SER, OUTPUT);
  pinMode(RCLK, OUTPUT);
  pinMode(SRCLK, OUTPUT);
}
```

Відтепер ми говоримо не про вивід 3, 5 чи 7, а про виводи **SER**, **RCLK** і **SRCLK**: перетворення робить машина, коли компілює програму і відправляє її в Arduino. Якщо ми на рядку 6522 програми з 8000 рядків і треба ввімкнути дріт, з'єднаний із SER, набагато легше пам'ятати **назву**, ніж число. До якого виводу він під'єднаний? Це написано один раз, **на початку файлу**.

Примітка

Замінювати можна **чим завгодно**, навіть цілою інструкцією: наприклад `#define ACCENDI_SER digitalWrite(SER, HIGH)` (*accendi* = "увімкни"), а потім у коді писати лише `ACCENDI_SER`. Якщо довести ідею до крайності, можна збудувати власну мову програмування з `#define`, але так **ніколи не роблять**: нам досить дати **назву кожному дроту**.

4.3 `#define` — це не змінна

Увага

`#define` **схожий** на змінну (ім'я, в якому лежить число), але це зовсім інше:

- **змінна** — це ім'я, яке **займає місце в пам'яті** (шухляду) і яке я можу **змінювати коли завгодно**, поки програма працює;
- `#define` — це **константа**: мітка, яку замінюють **до компіляції**, вона **не займає пам'яті** і її **не можна змінити** під час виконання (*runtime*). Змінити її можна лише у рядку на початку файлу, а потім треба **перекомпілювати**.

У C є **неписане правило**, якого дотримуються майже всі програмісти: **змінні** пишуть **малими літерами**, **константи** — **ВЕЛИКИМИ**. Якщо його не дотримуватися, програма все одно працює, але так той, хто бачить ім'я великими літерами, одразу знає, навіть не знаючи, де воно визначене, що це значення **не можна чіпати**.

5 Як завантажити дані в 595

5.1 Повторення: три сигнали

Керування 595 пояснено в матеріалі A04 з електроніки; нагадаємо три сигнали:

- **SER**: **вхід даних**, куди ми подаємо по одному біту;

- **SRCLK**: **тактовий** **сигнал**, який **заводить** біт із SER у регістр зсуву, зсуваючи на одне місце ті, що вже всередині;
- **RCLK**: сигнал, який **виводить** на виходи **одним** **махом** усе завантажене, щоб не було видно, як **біти** **пробігають** регістром.

Згадайте **трубку** **з** **кульками**: у трубки **вісім** **віконець**, і ми вкидаємо **червоні** кульки (світлодіод світиться, 1) або **білі** (світлодіод не світиться, 0). Щоб вкинути кульку, спершу **підносимо** її до отвору, вибравши колір (SER високий або низький), потім **заштовхуємо** (один імпульс SRCLK). Поки ми заповнюємо трубку, віконця **закриті**: ми відкриваємо їх (“робимо фото” через RCLK), лише коли всі кульки на місці.

5.2 Приклад: послідовність для завантаження

Вибираємо, які з 8 світлодіодів увімкнути. У прикладі з уроку, починаючи з **першої** **кульки,** **яку** **вкидаємо**: червона, біла, червона, біла, червона, червона, біла, червона, тобто `1 0 1 0 1 1 0 1`.

Увага

595 — це **регістр** **зсуву**: **перша** **кулька**, яку ми вкидаємо, в кінці опиняється **в** **глибині** трубки. Тому треба міркувати **навпаки**: якщо почати не з того світлодіода, послідовність буде **перевернутою**. Нічого страшного: досить **перевернути** **розташування** **світлодіодів** (або читати ряд з іншого боку).

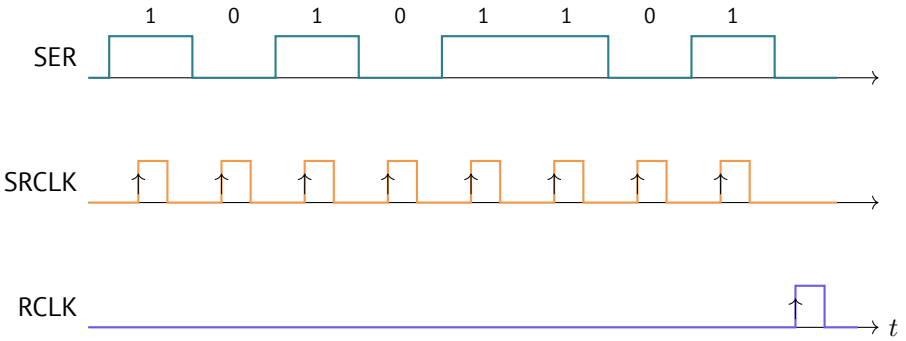


Рис. 5: Часова діаграма з дошки: на SER один за одним подаються 8 бітів `1 0 1 0 1 1 0 1`; кожному біту відповідає імпульс SRCLK (біт заходить на фронті наростання); у кінці один імпульс RCLK виводить результат на виходи. Коли ми закінчили завантаження, RCLK отримує свій імпульс, і лише тоді світлодіоди змінюються.

5.3 Перша версія: по одній кульці

Щоб вкинути першу кульку (червону), пишемо:

```
// підношу ЧЕРВОНУ кульку до трубки
digitalWrite(SER, HIGH);
delay(1);
// заштовхую її: піднімаю й опускаю SRCLK
digitalWrite(SRCLK, HIGH);
delay(1);
digitalWrite(SRCLK, LOW);
delay(1);
```

“Подай дані, потім підніми й опусти”: як **натиснути** **кнопку**, щоб зробити фото, а потім **прибрати** **палець**. Якби ми написали лише три інструкції `digitalWrite` одну за одною, Arduino виконав би їх так швидко, що 595 міг би **не** **встигнути** зрозуміти, що сталося. Для надійності ставимо **маленьку** **паузу** між операціями: `delay`. Треба зовсім трохи: **1** **мілісекунду**. 595 достатньо швидкий, щоб приймати навіть тисячу даних за секунду, і насправді можна було б іти **набагато** **швидше**; з нашими `delay(1)` (по три на кожен біт) ми завантажуємо кілька сотень бітів за секунду, цього більш ніж досить. Якщо хочемо **побачити** операцію спокійно, можна поставити навіть `delay(100)`.

Для другої кульки (білої) повторюємо ті самі рядки з `LOW` на SER, і так для всіх восьми. Це дуже довго: ми пишемо рядок за рядком для речі, яка мала б бути простою.

5.4 Друга версія: функція `carica()`

Помічаємо, що є шматок, який ми **повторюємо** **щоразу**, однаковий: підняти SRCLK, почекати, опустити SRCLK, почекати. Ми говорили про макроси: зберімо його у **функцію**. Вона нічого не повертає, отже вона **`void`**; назвемо її `carica` (“завантаж”; можна назвати і `clock`, бо це радше те, що вона робить).

```
void carica()
{
  digitalWrite(SRCLK, HIGH);
  delay(1);
  digitalWrite(SRCLK, LOW);
  delay(1);
}
```

Тепер кожна кулька потребує лише трьох рядків замість шести: ми **скоротили** **код** **удвічі**, а ці чотири рядки написали **лише** **один** **раз**, у функції. Після 8 бітів лишається ще одна операція, не така, як інші: імпульс **RCLK**.

```
#define SER 3
#define RCLK 5
#define SRCLK 7

void setup()
{
  pinMode(SER, OUTPUT);
  pinMode(RCLK, OUTPUT);
  pinMode(SRCLK, OUTPUT);
}

void loop()
{
  // вкидаю першу ЧЕРВОНУ кульку
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // вкидаю другу БІЛУ кульку
  digitalWrite(SER, LOW);
  delay(1);
  carica();

  // вкидаю третю ЧЕРВОНУ кульку
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // вкидаю четверту БІЛУ кульку
  digitalWrite(SER, LOW);
  delay(1);
  carica();

  // вкидаю пяту' ЧЕРВОНУ кульку
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // вкидаю шосту ЧЕРВОНУ кульку
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // вкидаю сьому БІЛУ кульку
  digitalWrite(SER, LOW);
  delay(1);
  carica();

  // вкидаю восьму ЧЕРВОНУ кульку
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // вмикаю RCLK, щоб перенести дані з регістра на виходи
  digitalWrite(RCLK, HIGH);
  delay(1);
  digitalWrite(RCLK, LOW);
  delay(1);
}

void carica()
{
  digitalWrite(SRCLK, HIGH);
  delay(1);
  digitalWrite(SRCLK, LOW);
  delay(1);
}
```

Примітка

Кожна кулька має `delay(1)` перед `carica()`: його можна прибрати звідти і поставити ще один **всередину** `carica()`, на початку. Працювало б так само, і це навіть кращий вибір; поки що залишаємо його зовні, бо розмова йде далі.

Примітка

В Arduino (і в Tinkercad) функцію на зразок `carica()` можна писати **під** `loop()`, який її використовує: середовище Arduino саме додає оголошення. У "звичайній" програмі на C її треба було б оголосити раніше.

5.5 Світлодіоди "вибухають": увага до струму

Коли копіюємо код у Tinkercad і запускаємо симуляцію, світлодіоди **вибухають**. Причина — **струм**: з резисторами 220 Ω кожен світлодіод споживає понад 10 мА, і коли 8 світлодіодів світяться разом, загальний струм через 595 перевищує той, який мікросхема здатна витримати. Рішення — **збільшити** **резистори**, наприклад до **470 Ω**: струм кожного світлодіода приблизно **зменшується** **вдвічі**, а схема для наших цілей працює так само.

Приклад

Розрахунок. На уроці ми зробили **оцінку** **усно**: приблизно 20 мА на світлодіод, отже приблизно 160 мА разом, забагато для мікросхеми; з 470 Ω струм приблизно зменшується вдвічі. Ось точніший розрахунок. Для червоного світлодіода (приблизно 2 В на ньому) і живлення 5 В на резисторі лишається приблизно 3 В.

- З 220 Ω: $I = 3\,\text{В}/220\,\Omega \approx 13{,}6\,\text{мА}$ на світлодіод, отже з 8 увімкненими світлодіодами приблизно **109 мА**.
- З 470 Ω: $I = 3\,\text{В}/470\,\Omega \approx 6{,}4\,\text{мА}$ на світлодіод, отже з 8 світлодіодами приблизно **51 мА**.

Даташит 74HC595 вказує максимальну межу **70 мА** загалом через виводи живлення (VCC або GND) і 35 мА на кожен окремий вихід: з 220 Ω і всіма світлодіодами ми її перевищуємо, з 470 Ω залишаємося нижче.

Після виправлення послідовність з'являється, але **навпаки** порівняно з тим, як ми її задумали: це ефект регістра зсуву (перша кулька опиняється в глибині), і досить читати ряд з іншого боку.

6 Третя версія: число і побітове AND

6.1 Число 181

Код усе ще довгий. Помічаємо, що блоки кульок **усі** **однакові**, крім `HIGH` або `LOW`. Нагадаємо, що **`HIGH`** у числах — це **1**, а **`LOW`** — це **0**. Якби ми знайшли **формулу**, яка обчислює 1 або 0 з числа, усі блоки можна було б написати однаково.

Запишімо послідовність світлодіодів як **двійкове** **число**. Пронумеруймо позиції **від** **0** **до** **7** справа: кулька, вкинута першою, — це **біт** **0**, остання — **біт** **7**. Читаючи від біта 7 до біта 0, отримуємо `10110101`. Щоб перевести в **десяткове**, кожній цифрі дають **вагу** $2^0, 2^1, \ldots, 2^7$ і додають лише ваги цифр, що дорівнюють 1:

позиція	7	6	5	4	3	2	1	0
вага	128	64	32	16	8	4	2	1
біт	1	0	1	1	0	1	0	1
додаємо	128	–	32	16	–	4	–	1

$128 + 32 + 16 + 4 + 1 = 181$. Наша послідовність світлодіодів — це **181, записане у двійковій системі**.

6.2 Для комп'ютера кожне число, відмінне від нуля, — "істина"

Визначення

Для комп'ютера **лише нуль — хибність** (низький рівень, вимкнено, `LOW`). **Будь-яке інше число — істина**: високий рівень, увімкнено, `HIGH`.

Отже `digitalWrite(3, 72)` **вмикає** вивід 3, бо 72 відмінне від нуля; `digitalWrite(3, -9)` теж його вмикає. Єдиний спосіб вимкнути — написати **0**. Тож нам потрібна операція, яка з 181 дає **нуль**, коли потрібний біт дорівнює 0, і **число, відмінне від нуля**, коли він дорівнює 1.

6.3 Побітове AND (&)

Оператор **AND** дає **1 лише якщо обидва** входи дорівнюють 1. **Побітове AND** (у C **&**, один амперсанд) застосовує його до **кожної пари бітів** двох чисел: по суті це як **множення** біт на біт. Цей оператор ми бачили минулого року.

Робимо AND між 181 і числом, у якому **лише одна 1** на позиції, яку хочемо прочитати:

```
  10110101 (181)          10110101 (181)
& 00000001              & 00000010
= 00000001 ≠ 0: світиться = 00000000 = 0: не світиться
```

У першому випадку результат відмінний від нуля: світлодіод вмикається, і саме цього ми хочемо (біт 0 числа 181 дорівнює 1). У другому випадку результат нуль: світлодіод не світиться, і саме цього ми хочемо (біт 1 числа 181 дорівнює 0). Отже для кожної кульки робимо AND між 181 і числом із **лише однією 1**, яка щоразу **змінює позицію**: перша кулька — перша позиція, друга кулька — друга позиція, і так до восьмої.

В Arduino двійкове число можна написати з **B** спереду, наприклад `B00000001`: це **8 цифр**, сім нулів і одна 1.

```
// вкидаю першу ЧЕРВОНУ кульку
digitalWrite(SER, 181 & B00000001);
delay(1);
carica();

// вкидаю другу БІЛУ кульку
digitalWrite(SER, 181 & B00000010);
delay(1);
carica();

// вкидаю третю ЧЕРВОНУ кульку
digitalWrite(SER, 181 & B00000100);
delay(1);
carica();
```

Ми написали 181 для простоти: це **те саме**, що `B10110101`, ми щойно це порахували.

Примітка

Запис `B00000001` — це зручність **Arduino**. У C++ (і в нових версіях C) двійкове число пишуть із префіксом `0b`, наприклад `0b00000001`, і Arduino теж його приймає.

6.4 Зсув уліво (<<)

Тепер кожен блок відрізняється від інших **лише позицією одиниці**. Чи можна написати `B00000010`, виходячи з `B00000001`? Так: **зсунути вліво** на одну позицію. Оператор **<<** (подвійне “менше”), який ми теж бачили минулого року, робить **зсув байта вліво**:

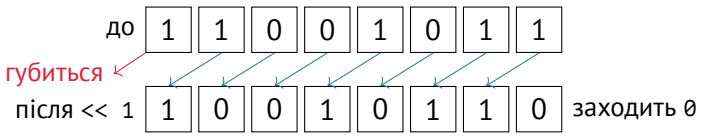


Рис. 6: Зсув уліво на одну позицію: кожен біт переходить на одне місце вліво, крайній лівий біт губиться, а справа завжди заходить 0. Зсув управо (>>) робить те саме у зворотному напрямку.

Отже:

- `B00000010` — це `(B00000001 << 1)`;
- `B00000100` — це `(B00000001 << 2)`; і так далі з 3, 4, 5, 6, 7;
- навіть перший можна записати за тією ж схемою: `(B00000001 << 0)`, тобто зсунутий на нуль позицій.

```
// вкидаю першу ЧЕРВОНУ кульку
digitalWrite(SER, 181 & (B00000001 << 0));
delay(1);
carica();

// вкидаю другу БІЛУ кульку
digitalWrite(SER, 181 & (B00000001 << 1));
delay(1);
carica();

// ... і так далі до восьмої
digitalWrite(SER, 181 & (B00000001 << 7));
delay(1);
carica();
```

Увага

Будьте уважні з **автодоповненням** редактора: коли копіюєте блоки, легко отримати неправильне початкове число. На уроці саме так і сталося: редактор доповнив `181 & (B00010000 << 4)` і `181 & (B00100000 << 5)`, тобто байт, **уже** **зсунутий**, зсувався **ще** **раз**, замість `B00000001 << 4` і `B00000001 << 5`. Зсувати треба **завжди** `B00000001`: змінюється лише **на** **скільки** ми його зсуваємо. З такою помилкою послідовність світлодіодів виходить не та, що очікувалась.

7 Остаточна версія: цикл `for`

А тепер найкраще. Блоки восьми кульок **усі** **однакові**, крім **одного** **числа** — величини зсуву, яка дорівнює 0, потім 1, 2, 3, … до 7. Шматок коду, який **повторюється** **8** **разів**, і щоразу число збільшується на 1: це саме робота для **циклу** **`for`**. Замість числа ставимо змінну **`i`**, яка першого разу дорівнює 0, потім 1, потім 2 і так далі. Усі вісім блоків викидаємо, і лишається кілька рядків:

```
#define SER 3
#define RCLK 5
#define SRCLK 7

void setup()
{
  pinMode(SER, OUTPUT);
  pinMode(RCLK, OUTPUT);
  pinMode(SRCLK, OUTPUT);
}

void loop()
{
  for (int i = 0; i < 8; i++)
  {
    digitalWrite(SER, 181 & (B00000001 << i));
    delay(1);
    carica();
  }

  // вмикаю RCLK, щоб перенести дані з регістра на виходи
  digitalWrite(RCLK, HIGH);
  delay(1);
  digitalWrite(RCLK, LOW);
  delay(1);
}

void carica()
{
  digitalWrite(SRCLK, HIGH);
  delay(1);
  digitalWrite(SRCLK, LOW);
  delay(1);
}
```

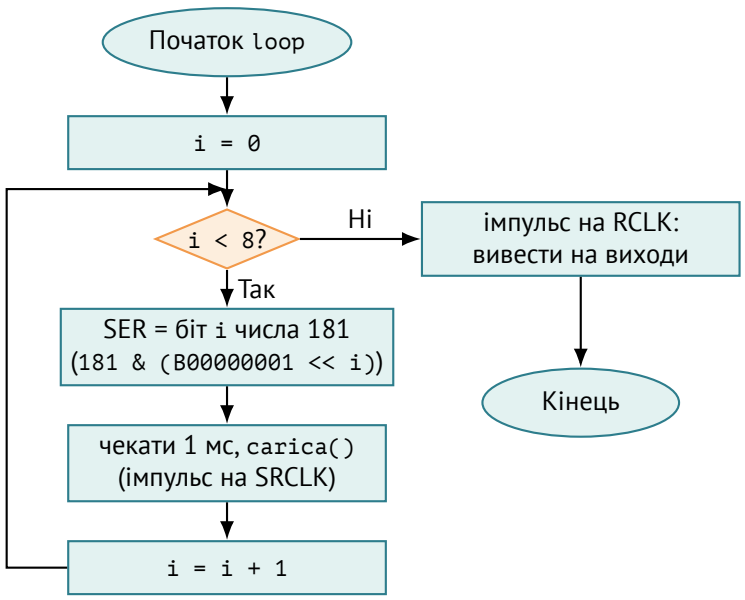


Рис. 7: Блок-схема одного проходу `loop()` в остаточній версії. Потім `loop()` починається знову, без кінця: “Кінець” позначає кінець одного проходу.

Код мав приблизно **70 рядків**, тепер приблизно **35**, і працює так само. Що краще? Ця версія: що **коротший** код, то менше шансів **помилитися** (це життєве правило програмістів). І є ще одна перевага: щоб показати іншу комбінацію світлодіодів, досить **змінити число 181**.

Увага

Типова помилка: якщо світяться (або не світяться) **усі світлодіоди**, мабуть, у `for` лишилося `<< 0` замість `<< i`. Тоді щоразу виводиться **той самий біт** (біт 0 числа 181, який дорівнює 1), а біт має **змінюватися залежно від позиції**. Інші типові помилки: **дроти, не вставлені** по-справжньому в отвори макетної плати, і **загублені рядки** при копіюванні коду через Ctrl+C / Ctrl+V. Завжди добре перевіряйте, що ви написали: якщо код працює на комп'ютері викладача, він має так само працювати і на вашому.

8 Що запам'ятати з цього уроку

- При **передачі за значенням** функція отримує **копію** параметра: зміни всередині функції **не змінюють** оригінальну змінну.
- Пам'ять — як нумерований **комод із шухлядами**: кожна змінна займає одну чи кілька шухляд (`char` 1 байт, `int` 2 байти на Arduino), які машина вибирає для нас **випадково**.
- У кінці функції її змінні **звільняються**: видаляється посилання, значення лишається, але шухляда знову вільна. Щоб отримати результат, використовують **`return`** і **присвоюють** його: `peso = calcolaPeso(lato);`.
- Дві змінні з **однаковим ім'ям**, одна в `loop`, друга у функції, — це **дві різні змінні**.
- Виводи мікросхеми нумерують, дивлячись **згори**: **вивід 1** ліворуч від **виїмки**, далі **проти годинникової стрілки**. У 595: 14 SER, 12 RCLK, 11 SRCLK, 10 SRCLR (до 5 В), 13 OE (до GND), 8 GND, 16 VCC.
- **`#define`** створює **мітку**, яку замінюють **до компіляції** (як "знайти і замінити"): це **константа**, вона не займає пам'яті і її не можна змінити під час роботи. Константи пишуть **ВЕЛИКИМИ** літерами, змінні — малими.
- Щоб завантажити біт у 595: подаємо біт на **SER**, даємо імпульс на **SRCLK** (функція `carica()`); після 8 бітів імпульс на **RCLK** виводить результат. Перший вкинутий біт опиняється **в глибині**.
- Коли 8 світлодіодів світяться разом, увага до **загального струму** через 595: з 470 Ω замість 220 Ω ми лишаємося в межах.
- Для комп'ютера **лише нуль — хибність**. `181 & (B00000001 << i)` дорівнює нулю, якщо біт `i` числа 181 дорівнює 0, і числу, відмінному від нуля, якщо він 1: у **`for`** від 0 до 7 це завантажує всі 8 бітів.

Приклад

До контрольної 29 жовтня: контрольна охоплює **увесь зміст роздаткових матеріалів**, **включно з цим**, і **firmware минулого року** (матеріали A01 і A01bis).

Глосарій: італійські слова, які вживають на уроці

Italiano	**Українською**
passaggio per valore	передача за значенням
copia, copiare	копія, копіювати
memoria, cassetto, cassettiera	пам'ять, шухляда, комод
variabile, deallocare	змінна, звільнити (пам'ять)
valore di ritorno (return)	значення, що повертається
peso, peso specifico, volume, lato	вага, питома вага, об'єм, сторона
piedino, tacca	вивід (pin), виїмка
senso antiorario	проти годинникової стрілки
etichetta, costante	мітка, константа
trova e sostituisci	знайти і замінити
registro a scorrimento	регістр зсуву
pallina rossa / bianca	червона / біла кулька
carica, caricare	завантаж, завантажувати
acceso / spento	світиться (увімкнено) / вимкнено
spostare a sinistra (shift)	зсунути вліво
corrente, resistore	струм, резистор
verifica	контрольна